

```
-- Elizabeth Gasparim and Irena Swanson
-- Implementation in Macaulay2, Version 0.9:
--   1. run Macaulay2
--   2. type: load "instanton.m2" -- the name of the file in quotes
--   3. run functions below
--
-- The main functions are:
--   iwidth(p,j)
--   iheight(p,j)
-- where p is a polynomial in x and y with rational/integer coefficients
-- and j is a nonnegative integer.
-- The two functions compute the instanton widths and heights
-- of the vector bundle on the blowup of  $\mathbb{C}^2$ 
-- whose transition matrix in canonical coordinates is of the form
--
--      |      z^j      p'      |
--      |      0      z^{-j}    |
--
-- where p' is the sum of those terms  $u^i z^l$  in  $p(u, zu)$  for which
--  $1 \leq i \leq 2j-2$  and  $0 \leq l \leq j-1$ .
--
-- Polynomials in Macaulay2 are typed as  $3*x^2*y^{15} - 2*x^3$ 
--
-- Two other self-explanatory functions:
--   Milnor(p)
--   Tjurina(p)
-- and a shortcut to compute all:
--   doall(p,j)
-- which returns {Milnor(p), Tjurina(p), iwidth(p,j), iheight(p,j)}
```

```
basing = ZZ/101
basing = QQ
S = basing[x,y]
uuu = x^4-x*y^5
vvv = x^4-x^2*y^3-x^2*y^5-y^8
ttt = x^3-y^4-y^5
p = x^2-y^3
q = y^3+x^4-x*y
```

```
ringparams = (p, j) -> (
  pp := {};
  if p == 0 then (
    m := 0;
    N := j;
  ) else (
    pp = terms(p);
    m = min apply(pp, i -> (degree i)_0);
    N = max apply(pp, i -> (exponents i)_0_0+(exponents i)_0_1) + 2*j-2;
  );
  return {m, N}
)
```

```
ldoring = (p, j, m, N) -> (
  aindex := flatten apply(m, i -> apply(i-j+1, l -> (i,l)));
  aind := toList flatten apply(m..N, i -> apply(i+1, l -> (i,l)));
  aindex = aindex | aind;
  avars := apply(aindex, i -> a_i);
  bindex := flatten apply(N+1, i -> toList apply(0..i+j, l -> (i,l)));
  bvars := apply(bindex, i -> b_i);
  R := basing[u,z,reverse(bvars | avars)];
  apoly := sum apply(aindex, i -> a_i*u^(i_0)*z^(i_1));
  bpoly := sum apply(bindex, i -> b_i*u^(i_0)*z^(i_1));
  F := map(R,S,{u,z*u});
  allvars := apply(numgens R - 2, i -> R_(i+2));
  aindex = select(aindex, i -> (i_0 > 2*j-2));
```

```

bindex = select(bindex, i -> (i_0 > 2*j-2));
irrelvars := apply(aindex, i -> a_i) | apply(bindex, i -> b_i);
return {allvars, F(p), apoly, bpoly, ideal (irrelvars | {0_R})}
)

getrels = (N, fTv) -> (
  R := ring fTv;
  nonfree := ideal(0_R);
  rels := ideal(0_R);
  k := 0;
  while (k <= N) do (
    -- truncate first entry of image in the neighborhood V:
    tempoly := fTv % u^(k+1);
    while tempoly != 0 do (
      tempterm := leadMonomial tempoly;
      i := (exponents tempterm)_0; -- u exponent
      l := (exponents tempterm)_1; -- z exponent
      partp := tempoly - (tempoly % (u^i*z^l));
      tempoly = tempoly - partp;
      fTv = fTv - partp;
      partp = partp/(u^i*z^l);
      if l > i then (
        ttempterm := leadTerm(partp);
        nonfree = nonfree + ideal(ttempterm);
        partp = partp/leadCoefficient(ttempterm);
        rels = rels + ideal(partp);
      );
    );
    k = k + 1;
  );
  return {nonfree, rels}
)

RtoS = (p) -> (
  g := 0_S;
  while p != 0 do (
    lf := leadTerm p;
    lff := exponents lf;
    i := lff_0_0; -- u exponent
    l := lff_0_1; -- z exponent
    if l <= i then
      g = g + y^l*x^(i-l)*leadCoefficient(lf);
    p = p - lf;
  );
  g
)

setvectors=(aapoly, bbpoly, changeables, allvars) -> (
  Smodule := image matrix{{0_S}, {0_S}};
  tozero = apply(allvars, i -> i => 0);
  total := #changeables;
  k := 0;
  while k < total do (
    tapoly := substitute(aapoly, changeables_k => 1);
    tbpoly := substitute(bbpoly, changeables_k => 1);
    tapoly = substitute(tapoly, tozero);
    tbpoly = substitute(tbpoly, tozero);
    vecab = matrix{{RtoS(tapoly)}, {RtoS(tbpoly)}};
    Smodule = Smodule + image vecab;
    k = k + 1;
  );
  presentation image mingens Smodule
)

qlength = (A) -> (

```

```

B = kernel transpose A; --dual coker A;
Q = (kernel transpose presentation B)/(image transpose gens B);
degree Q
)

```

```

iwidth = (p,j) -> (
  paramlist := ringparams(p,j);
  m := paramlist_0;
  N := paramlist_1;
  ringuzab := ldoring(p, j, m, N);
  allvars := ringuzab_0;
  pp := ringuzab_1;
  apoly := ringuzab_2;
  bpoly := ringuzab_3;
  irrelvarideal := ringuzab_4;
  fTv = z^j * apoly + pp* bpoly;
  relslist := getrels(N, fTv);
  nonfree := relslist_0 + irrelvarideal;
  rels := relslist_1;
  templist := flatten entries mingens rels;
  dosubst := apply(templist, i -> {leadTerm i => (leadTerm i) - i});
  tempnum := #dosubst;
  i := 0;
  while (i < tempnum) do (
    apoly = substitute(apoly, dosubst_i);
    bpoly = substitute(bpoly, dosubst_i);
    i = i + 1;
  );
  gb nonfree;
  changeables := apply(allvars, i -> i % nonfree);
  changeables = (entries mingens ideal changeables)_0;
  --A := setvectors(u^(j+N)*apoly, u^(j+N)*bpoly, changeables, allvars);
  A := setvectors(u^j*apoly, u^j*bpoly, changeables, allvars);
  qlength A
)

```

```

hdoring = (p, j, m, M) -> (
  cohct := 0;
  if (j > (m+1)/2) then (
    cohct = m*(j - (m+1)/2);
    if (m > j-2) then return (0,{},{},cohct);
    aindex := toList apply(m..j-2, i -> toList apply(i-j+1..-1, l -> (i,l)));
  ) else
    aindex = toList apply(j-1, i -> toList apply(i-j+1..-1, l -> (i,l)));
  aindex = reverse flatten aindex;
  avars := apply(aindex, i -> a_i);
  N := max(0, 2*j - 2) + 1;
  dindex := apply(max(0, N - m), i -> toList apply(i+1..i+j, l -> (i,l)));
  dindex = reverse flatten dindex;
  dvars := apply(dindex, i -> d_i);
  cindex := toList apply(m..N,i->toList apply(max(0,i-j-1)..M+N-m-1,l->(i,l)));
  cindex = reverse flatten cindex;
  cvars := apply(cindex, i -> c_i);
  T := basering[u, z, avars, cvars, dvars];
  apoly := sum apply(aindex, i -> a_i*u^(i_0)*z^(i_1 + j));
  cpoly := sum apply(cindex, i -> c_i*u^(i_0)*z^(i_1 + j));
  dpoly := sum apply(dindex, i -> d_i*u^(i_0)*z^(i_1));
  numa := #aindex;
  numcd := #cindex + #dindex;
  cdvars = take(gens T, -numcd);
  avars = take(take(gens T, numa+2), -numa);
  F := map(T,S,{u,z*u});
  pp := F(p);
  return {(apoly + cpoly + pp*dpoly) % u^N, avars, cdvars, cohct}
)

```

```

isbound = (f) -> (
  cobounds := {};
  while f != 0 do (
    lmf := leadMonomial f;
    i := (exponents lmf)_0; -- u exponent
    l := (exponents lmf)_1; -- z exponent
    fuz := substitute((f // (u^i*z^l)), {u => 0, z => 0})*u^i*z^l;
    -- fuz is the part of f of (u,z) degree (i,l)
    f = f - fuz;
    if l > i then (
      fuz = fuz // (u^i*z^l);
      -- fuz is now linear, does not involve u and z
      lmf = leadTerm(fuz);
      lcf = leadCoefficient(lmf);
      -- lmf is lcf times a variable
      -- this variable is forced to make fuz zero:
      intermsubst = {lmf // lcf => (lmf - fuz) // lcf};
      f = substitute(f, intermsubst);
      cobounds = apply(cobounds, i-> substitute(i,intermsubst));
      cobounds = cobounds | {fuz};
    );
  );
  cobounds
)

countdim = (cobounds, avars, cdvars) -> (
  cdtozero := apply(cdvars, i -> i => 0);
  atozero := apply(avars, i -> i => 0);
  tempbounds := {0_(ring ideal(cdvars_0))};
  i := 0;
  while (i < (#cobounds)) do (
    if (substitute(cobounds_i, atozero) != cobounds_i) then
      if (substitute(cobounds_i, cdtozero) != cobounds_i) then
        tempbounds = tempbounds | {substitute(cobounds_i, atozero)};
    i = i + 1;
  );
  return (#avars - codim ideal tempbounds)
)

iheight = (p,j) -> (
  if (p == 0_S) then return j*(j-1)/2;
  paramlist := ringparams(p,j);
  m := paramlist_0;
  M := j*(j-1)/2;
  if (j > m+1) then M = M - (j-m)*(j-m-1)/2;
  return M
)

Milnor = (p) -> (
  jac = minors(1,jacobian ideal (p));
  if codim jac < dim S then return "Milnor number not defined";
  Mideal = jac : saturate(jac);
  degree coker gens Mideal
)

Tjurina = (p) -> (
  jac = minors(1,jacobian ideal (p));
  if codim jac < dim S then return "Tjurina number not defined";
  Mideal = (jac + ideal(p)) : saturate((jac + ideal(p)));
  degree coker gens Mideal
)

doall = (p,j) -> (

```

```
{Milnor(p), Tjurina(p), iwidth(p,j), iheight(p,j)}  
)
```